```
import pandas as pd
import numpy as np
```

Zadania podstawowe

Zadanie1

```
daty=pd.date_range(start='2020-03-01',end='2020-03-05')
df1=pd.DataFrame(np.random.rand(5,3),columns=['A','B','C',],index=daty)
print(df1)
```

```
                   A         B         C
2020-03-01  0.770937  0.203661  0.479573
2020-03-02  0.952880  0.454678  0.441527
2020-03-03  0.923915  0.266564  0.211727
2020-03-04  0.596965  0.817009  0.381483
2020-03-05  0.483686  0.225243  0.364745
```

Zadanie2

```
df2=pd.DataFrame(np.random.randint(1,10,(20,3)),columns=['A','B','C'])
df2.index.name = 'id'
print(df2.head(3))
```

```
    A  B  C
id
0   5  7  1
1   3  1  9
2   2  2  9
```

```
print(df2.tail(3))
```

```
    A  B  C
18  6  8  5
19  7  3  9
20  3  3  8
```

nazwa indeksu tabeli

```
print(df2.index.name)
```

```
id
```

nazwy wszystkich kolumn

```
print(df2.columns)
```

```
Index(['A', 'B', 'C'], dtype='object')
```

jedynie wartosci tabel bez indeksow i naglowkow

```
print(df2.values)
```

```
[[5 7 1]
 [3 1 9]
 [2 2 9]
 [7 1 3]
 [3 6 6]
 [8 6 1]
 [9 6 2]
 [1 9 4]
 [6 3 2]
 [4 3 1]
 [6 1 2]
 [1 3 9]
 [6 5 5]
 [7 2 5]
 [2 3 4]
 [7 8 2]
 [1 2 1]
 [9 4 9]
 [8 7 8]
 [3 2 9]]
```

piec losowych wierszy z tabeli

```
print(df2.sample(5))
```

```
    A  B  C
id
14  2  3  4
8   6  3  2
15  7  8  2
9   4  3  1
5   8  6  1
```

Zadanie3

```
print(df2['A'].tolist())
```

```
[5, 3, 2, 7, 3, 8, 9, 1, 6, 4, 6, 1, 6, 7, 2, 7, 1, 9, 8, 3]
```

```
print(df2[['A','B']])
```

```
    A  B
id
0   5  7
1   3  1
```

```
2   2  2
3   7  1
4   3  6
5   8  6
6   9  6
7   1  9
8   6  3
9   4  3
10  6  1
11  1  3
12  6  5
13  7  2
14  2  3
15  7  8
16  1  2
17  9  4
18  8  7
19  3  2
```

```
print(df2[['A','B']].values.tolist())
```

```
[[5, 7], [3, 1], [2, 2], [7, 1], [3, 6], [8, 6], [9, 6], [1, 9], [6, 3], [4,
```

.values wyciagniecie danych i zmiana na tablice numpy

Zadanie4

```
print(df2.iloc[0:3,0:2])
```

```
    A  B
id
0   5  7
1   3  1
2   2  2
```

```
print(df2.iloc[5])
```

```
A    8
B    6
C    1
Name: 5, dtype: int64
```

```
print(df2.iloc[[0,5,6,7],[1,2]])
```

```
    B  C
id
0   7  1
5   6  1
6   6  2
7   9  4
```

Zadanie5

```
print(df2.describe())
```

```
                A          B          C
count  20.000000  20.000000  20.000000
mean    4.900000   4.050000   4.600000
std     2.751076   2.502104   3.185493
min     1.000000   1.000000   1.000000
25%     2.750000   2.000000   2.000000
50%     5.500000   3.000000   4.000000
75%     7.000000   6.000000   8.250000
max     9.000000   9.000000   9.000000
```

```
print(df2>0)
#print((df2>0).describe())
```

```
       A     B     C
id
0   True  True  True
1   True  True  True
2   True  True  True
3   True  True  True
4   True  True  True
5   True  True  True
6   True  True  True
7   True  True  True
8   True  True  True
9   True  True  True
10  True  True  True
11  True  True  True
12  True  True  True
13  True  True  True
14  True  True  True
15  True  True  True
16  True  True  True
17  True  True  True
18  True  True  True
19  True  True  True
```

```
print(df2['A'][df2['A']>0])
```

```
id
0     5
1     3
2     2
3     7
4     3
5     8
6     9
7     1
8     6
9     4
10    6
11    1
12    6
13    7
14    2
15    7
16    1
```

```
17    9
18    8
19    3
Name: A, dtype: int64
```

Zadanie6

dla kolumny

```
print(df2.mean())
```

```
A    4.90
B    4.05
C    4.60
dtype: float64
```

dla wierszy axis=1 ->poziomo

```
print(df2.mean(axis=1))
```

```
id
0     4.333333
1     4.333333
2     4.333333
3     3.666667
4     5.000000
5     5.000000
6     5.666667
7     4.666667
8     3.666667
9     2.666667
10    3.000000
11    4.333333
12    5.333333
13    4.666667
14    3.000000
15    5.666667
16    1.333333
17    7.333333
18    7.666667
19    4.666667
dtype: float64
```

zadanie7

```
df3=pd.DataFrame(np.random.randint(1,10,(20,3)),columns=['A','B','C'])
res=pd.concat([df2,df3])
print(res)
```

```
    A  B  C
0   5  7  1
1   3  1  9
2   2  2  9
```

```
3   7  1  3
4   3  6  6
5   8  6  1
6   9  6  2
7   1  9  4
8   6  3  2
9   4  3  1
10  6  1  2
11  1  3  9
12  6  5  5
13  7  2  5
14  2  3  4
15  7  8  2
16  1  2  1
17  9  4  9
18  8  7  8
19  3  2  9
0   8  7  1
1   9  8  8
2   4  5  6
3   5  7  9
4   9  1  9
5   1  6  7
6   7  2  6
7   6  8  6
8   2  8  4
9   9  7  6
10  2  7  7
11  5  4  4
12  7  3  2
13  9  5  9
14  9  8  6
15  9  4  3
16  2  4  2
17  5  8  8
18  2  5  4
19  2  7  9
```

```
print(res.transpose())
```

```
   0   1   2   3   4   5   6   7   8   9   ...  10  11  12  13  14  15  16  \
A   5   3   2   7   3   8   9   1   6   4  ...   2   5   7   9   9   9   2
B   7   1   2   1   6   6   6   9   3   3  ...   7   4   3   5   8   4   4
C   1   9   9   3   6   1   2   4   2   1  ...   7   4   2   9   6   3   2

   17  18  19
A   5   2   2
B   8   5   7
C   8   4   9

[3 rows x 40 columns]
```

Zadanie8

```
data ={
 'Day': ['Mon ','Mon ','Tue ','Tue '] ,
```

```
 'Fruit': ['Apple ','Banana ','Apple ','Banana '] ,
 'Sales': [10 ,12 ,9 ,7]
 }
df4=pd.DataFrame(data)
```

```
print(df4.groupby('Day')['Sales'].sum())
```

```
Day
Mon     22
Tue     16
Name: Sales, dtype: int64
```

```
print(df4.groupby(['Day','Fruit'])['Sales'].sum())
```

```
Day   Fruit
Mon   Apple      10
      Banana     12
Tue   Apple       9
      Banana      7
Name: Sales, dtype: int64
```

Zadanie10

```
res['B']=1
print(res)
```

```
    A  B  C
0   5  1  1
1   3  1  9
2   2  1  9
3   7  1  3
4   3  1  6
5   8  1  1
6   9  1  2
7   1  1  4
8   6  1  2
9   4  1  1
10  6  1  2
11  1  1  9
12  6  1  5
13  7  1  5
14  2  1  4
15  7  1  2
16  1  1  1
17  9  1  9
18  8  1  8
19  3  1  9
0   8  1  1
1   9  1  8
2   4  1  6
3   5  1  9
4   9  1  9
5   1  1  7
6   7  1  6
7   6  1  6
8   2  1  4
```

```
9   9  1  6
10  2  1  7
11  5  1  4
12  7  1  2
13  9  1  9
14  9  1  6
15  9  1  3
16  2  1  2
17  5  1  8
18  2  1  4
19  2  1  9
```

zmiana wszystkich wartosci w kolumnie B na 1

```
res.iloc[1,2]=10
print(res)
```

```
    A  B   C
0   5  1   1
1   3  1  10
2   2  1   9
3   7  1   3
4   3  1   6
5   8  1   1
6   9  1   2
7   1  1   4
8   6  1   2
9   4  1   1
10  6  1   2
11  1  1   9
12  6  1   5
13  7  1   5
14  2  1   4
15  7  1   2
16  1  1   1
17  9  1   9
18  8  1   8
19  3  1   9
0   8  1   1
1   9  1   8
2   4  1   6
3   5  1   9
4   9  1   9
5   1  1   7
6   7  1   6
7   6  1   6
8   2  1   4
9   9  1   6
10  2  1   7
11  5  1   4
12  7  1   2
13  9  1   9
14  9  1   6
15  9  1   3
16  2  1   2
17  5  1   8
18  2  1   4
19  2  1   9
```

zmiana wartosci z 2 wiersza 3 kolumny na 10 (INDEKS 1 C)

```
res[res<0]=-res
print(res)
```

```
    A  B   C
0   5  1   1
1   3  1  10
2   2  1   9
3   7  1   3
4   3  1   6
5   8  1   1
6   9  1   2
7   1  1   4
8   6  1   2
9   4  1   1
10  6  1   2
11  1  1   9
12  6  1   5
13  7  1   5
14  2  1   4
15  7  1   2
16  1  1   1
17  9  1   9
18  8  1   8
19  3  1   9
0   8  1   1
1   9  1   8
2   4  1   6
3   5  1   9
4   9  1   9
5   1  1   7
6   7  1   6
7   6  1   6
8   2  1   4
9   9  1   6
10  2  1   7
11  5  1   4
12  7  1   2
13  9  1   9
14  9  1   6
15  9  1   3
16  2  1   2
17  5  1   8
18  2  1   4
19  2  1   9
```

zmiana znaku ujemnych liczb na dodatni

ZADANIA PODSUMOWUJACE

Zad11

```
df5=pd.DataFrame(np.random.randint(-100,100,(100,4)),columns=['A','B','C','D
```

```
warunek=((df5['A']>0) & (df5['B']<0))
df6=df5[warunek]
print(df6)
```

```
     A   B    C   D
3   59 -92   17  51
5   29 -62   36  68
13  83 -73   59  64
15  60 -11   62 -90
24  86 -78 -100  92
25  97 -70   97 -32
26  59  -4  -79  29
27  66 -40  -53 -27
28  54 -24   56 -78
43  59 -35   20  47
47  10 -95  -39  81
48  58 -44   78  77
50  62 -90  -74 -11
52  15 -98   46 -21
58  67 -77  -85  -9
65  47  -3   -2  15
73  73 -27  -94  26
78  58 -60  -28  53
81  42 -17  -76 -21
90  71 -87   35  -5
91  51 -94  -62  39
93  78 -75  -64 -38
```

```
print(df6.mean)
```

```
<bound method DataFrame.mean of      A   B    C   D
3   59 -92   17  51
5   29 -62   36  68
13  83 -73   59  64
15  60 -11   62 -90
24  86 -78 -100  92
25  97 -70   97 -32
26  59  -4  -79  29
27  66 -40  -53 -27
28  54 -24   56 -78
43  59 -35   20  47
47  10 -95  -39  81
48  58 -44   78  77
50  62 -90  -74 -11
52  15 -98   46 -21
58  67 -77  -85  -9
65  47  -3   -2  15
73  73 -27  -94  26
78  58 -60  -28  53
81  42 -17  -76 -21
90  71 -87   35  -5
91  51 -94  -62  39
93  78 -75  -64 -38>
```

Zadanie12

```
products=pd.DataFrame({
    'ProductID':[1,2,4,5,5],
    'Name':['A','B','C','D','E']
})
sales=pd.DataFrame({'ProductID':[1,1,3,4,1],'Sales':[1,2,3,4,5]
    })
```

```
salesandproducts=products.merge(sales)
print(salesandproducts)
```

```
   ProductID Name  Sales
0          1    A      1
1          1    A      2
2          1    A      5
3          4    C      4
```

```
salesandproducts=products.merge(sales,how='right')
print(salesandproducts)
```

```
   ProductID Name  Sales
0          1    A      1
1          1    A      2
2          3  NaN      3
3          4    C      4
4          1    A      5
```

right to wszystkie sprzedaze

```
salesandproducts=products.merge(sales,how='left')
print(salesandproducts)
```

```
   ProductID Name  Sales
0          1    A    1.0
1          1    A    2.0
2          1    A    5.0
3          2    B    NaN
4          4    C    4.0
5          5    D    NaN
6          5    E    NaN
```

wszystkie produkty

```
salesandproducts=products.merge(sales,how='inner')
print(salesandproducts)
```

```
   ProductID Name  Sales
0          1    A      1
1          1    A      2
2          1    A      5
3          4    C      4
```

tylko dopasowane

Zadanie13

```
data1 =pd.DataFrame({
 'Day': ['Mon ','Mon ','Tue ','Tue '] ,
 'Fruit': ['Apple ','Banana ','Apple ','Banana '] ,
 'Sales': [10 ,12 ,9 ,7]
 })
```

```
print(pd.pivot_table(data1,values='Sales',index='Day',aggfunc='sum'))
```

```
      Sales
Day
Mon      22
Tue      16
```

```
print(pd.pivot_table(data1,values='Sales',index='Day',aggfunc='mean'))
```

```
      Sales
Day
Mon    11.0
Tue     8.0
```

Zadanie14

```
d= pd.DataFrame(np.random.rand(5, 3),columns=['A', 'B', 'C']
)
d.index.name='nazwa'
print(d)
```

```
               A         B         C
nazwa
0      0.975623  0.808003  0.991765
1      0.863658  0.493706  0.274622
2      0.505168  0.357502  0.712954
3      0.901535  0.619703  0.869809
4      0.059115  0.737717  0.661626
```

```
d=d.set_index('A')
print(d)
```

```
                B         C
A
0.975623  0.808003  0.991765
0.863658  0.493706  0.274622
0.505168  0.357502  0.712954
0.901535  0.619703  0.869809
0.059115  0.737717  0.661626
```

```
d=d.reset_index()
print(d)
```

```
          A         B         C
0  0.975623  0.808003  0.991765
1  0.863658  0.493706  0.274622
2  0.505168  0.357502  0.712954
3  0.901535  0.619703  0.869809
4  0.059115  0.737717  0.661626
```

Zadanie15

```
d['D']=d['A']+d['B']
print(d)
```

```
          A         B         C         D
0  0.975623  0.808003  0.991765  1.783626
1  0.863658  0.493706  0.274622  1.357364
2  0.505168  0.357502  0.712954  0.862670
3  0.901535  0.619703  0.869809  1.521238
4  0.059115  0.737717  0.661626  0.796833
```

```
d['meanABC'] = (d['A'] + d['B'] + d['C']) / 3
print(d)
```

```
          A         B         C         D   meanABC
0  0.975623  0.808003  0.991765  1.783626  0.925130
1  0.863658  0.493706  0.274622  1.357364  0.543995
2  0.505168  0.357502  0.712954  0.862670  0.525208
3  0.901535  0.619703  0.869809  1.521238  0.797016
4  0.059115  0.737717  0.661626  0.796833  0.486153
```

```
d['meanABC']=d[['A','B','C']].mean(axis=1)
print(d)
```

```
          A         B         C         D   meanABC
0  0.975623  0.808003  0.991765  1.783626  0.925130
1  0.863658  0.493706  0.274622  1.357364  0.543995
2  0.505168  0.357502  0.712954  0.862670  0.525208
3  0.901535  0.619703  0.869809  1.521238  0.797016
4  0.059115  0.737717  0.661626  0.796833  0.486153
```